

Sesión 3: Regresión Lineal y Logística

De predecir números a clasificar categorías

Agustín Gurvich

Introducción a Machine Learning

Parte I

Regresión lineal

¿Qué es regresión lineal?

Definición

Un modelo que predice un valor **numérico continuo** a partir de una combinación lineal de las variables de entrada.

Ejemplo

Precio de una casa a partir de sus m^2 , cantidad de ambientes y antigüedad. El precio puede ser cualquier número, no una categoría.

Es el primer modelo que vamos a entrenar a full.

La recta que mejor ajusta

Con una sola variable de entrada x , buscamos la recta

$$\hat{y} = wx + b$$

- w : pendiente, cuánto cambia $\hat{y}$ por cada unidad de x
- b : ordenada al origen, valor de $\hat{y}$ cuando $x = 0$

“Entrenar” el modelo es eso: encontrar los valores de w y b que mejor ajustan a los datos.

¿Qué significa “mejor ajusta”?

Definición

El **error cuadrático medio** (MSE) mide qué tan lejos están las predicciones del modelo de los valores reales.

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Elevamos al cuadrado para que errores positivos y negativos no se cancelen entre sí, y para penalizar más los errores grandes. Entrenar es buscar el w y el b que minimizan el MSE.

¿Cómo encontramos el mínimo?

- 1 Empezar con w y b al azar (predicciones malas)
- 2 Medir el error
- 3 Mover w y b un poquito en la dirección que lo reduce
- 4 Repetir hasta que el error deje de bajar

¿Cómo encontramos el mínimo?

- 1 Empezar con w y b al azar (predicciones malas)
- 2 Medir el error
- 3 Mover w y b un poquito en la dirección que lo reduce
- 4 Repetir hasta que el error deje de bajar

Ejemplo

¿Y cómo salimos de este pozo? Pues cavando.

Learning rate

- Controla el tamaño de cada paso del descenso de gradiente
- Muy chico: converge, pero tarda muchísimas iteraciones
- Muy grande: puede pasarse del mínimo y no converger nunca

- Controla el tamaño de cada paso del descenso de gradiente
- Muy chico: converge, pero tarda muchísimas iteraciones
- Muy grande: puede pasarse del mínimo y no converger nunca

Importante

Nada de esto se programa a mano: scikit-learn ya lo resuelve, y de forma más eficiente que cualquier loop que escribamos nosotros.

Más de una variable

En un problema real casi siempre hay varias features. La idea es la misma, con un coeficiente por cada una:

$$\hat{y} = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

Ejemplo

$$\text{Precio} = w_1 \cdot \text{m}^2 + w_2 \cdot \text{ambientes} + w_3 \cdot \text{antigüedad} + b$$

Interpretando los coeficientes

- Coeficiente grande y positivo: esa variable empuja fuerte la predicción para arriba
- Coeficiente negativo: la empuja para abajo
- Coeficiente cercano a 0: esa variable casi no influye

Interpretando los coeficientes

- Coeficiente grande y positivo: esa variable empuja fuerte la predicción para arriba
- Coeficiente negativo: la empuja para abajo
- Coeficiente cercano a 0: esa variable casi no influye

Ojo: si las variables están en escalas muy distintas, comparar coeficientes a simple vista engaña.
Por eso escalamos antes de entrenar!

R^2 : coeficiente de determinación

Definición

Mide qué proporción de la variabilidad de y explica el modelo. Va de $-\infty$ a 1.

- $R^2 = 1$: predicciones perfectas
- $R^2 = 0$: el modelo predice tan bien como usar siempre el promedio de y
- $R^2 < 0$: el modelo es peor que predecir el promedio

Regresión lineal con scikit-learn

```
from sklearn.linear_model import LinearRegression

modelo = LinearRegression()
modelo.fit(X_train, y_train)

modelo.coef_          # los  $w_i$ 
modelo.intercept_     # el  $b$ 

modelo.predict(X_test)
modelo.score(X_test, y_test)  #  $R^2$ 
```

Parte II

Regresión logística

El problema: clasificar, no predecir un número

Ejemplo

Si un mail es spam o no, si un tumor es maligno o benigno, si un cliente deja de pagar o no. La salida es una **categoría**, no un número continuo.

¿Por qué no usar regresión lineal directamente, prediciendo 0 o 1?

El problema: clasificar, no predecir un número

Ejemplo

Si un mail es spam o no, si un tumor es maligno o benigno, si un cliente deja de pagar o no. La salida es una **categoría**, no un número continuo.

¿Por qué no usar regresión lineal directamente, prediciendo 0 o 1?

Importante

Porque puede devolver cualquier número: -3 , $1,7$, 250 . Eso no se interpreta como “sí” o “no”, y un solo outlier corre toda la recta.

Definición

Aplasta cualquier número real al rango $(0, 1)$.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- $z \rightarrow +\infty \Rightarrow \sigma(z) \rightarrow 1$
- $z \rightarrow -\infty \Rightarrow \sigma(z) \rightarrow 0$
- $z = 0 \Rightarrow \sigma(z) = 0,5$

Forma de “S”: un número cualquiera entra, algo interpretable como probabilidad sale.

Regresión lineal, pasada por la sigmoide:

$$\hat{p} = \sigma(w_1x_1 + \cdots + w_nx_n + b)$$

Definición

$\hat{p}$ se interpreta como la **probabilidad** de que la observación sea de la clase positiva (por ejemplo, “es spam”).

De probabilidad a clase: el umbral

- $\hat{p} \geq 0,5$: predecimos clase 1
- $\hat{p} < 0,5$: predecimos clase 0

De probabilidad a clase: el umbral

- $\hat{p} \geq 0,5$: predecimos clase 1
- $\hat{p} < 0,5$: predecimos clase 0

Importante

0,5 es el umbral por defecto, no una ley física. En diagnóstico médico, por ejemplo, conviene bajarlo para no dejar pasar casos positivos, aunque eso traiga más falsos positivos.

¿Cómo se entrena?

Definición

La función de costo acá es la **log loss**: penaliza fuerte cuando el modelo está muy seguro y se equivoca.

Ejemplo

Predecir $\hat{p} = 0,99$ para algo que en realidad es clase 0 sale mucho más caro que predecir $\hat{p} = 0,6$ y fallar igual.

¿Y si hay más de dos categorías?

- **One-vs-rest**: un clasificador binario por clase (“es esta” vs. “no lo es”), se queda con el de mayor probabilidad
- **Softmax**: generaliza la sigmoide a varias clases a la vez, con probabilidades que suman 1

scikit-learn elige la estrategia solo, según el problema.

Regresión logística con scikit-learn

```
from sklearn.linear_model import LogisticRegression

modelo = LogisticRegression()
modelo.fit(X_train, y_train)

modelo.predict(X_test)           # clases predichas
modelo.predict_proba(X_test)     # probabilidad por clase

modelo.score(X_test, y_test)     # accuracy
```

Regresión logística con scikit-learn

```
from sklearn.linear_model import LogisticRegression

modelo = LogisticRegression()
modelo.fit(X_train, y_train)

modelo.predict(X_test)           # clases predichas
modelo.predict_proba(X_test)     # probabilidad por clase

modelo.score(X_test, y_test)     # accuracy
```

`predict_proba` es la gran diferencia frente a regresión lineal: no solo da una clase, da qué tan segura está.

- Regresión lineal: ajusta una recta (o hiperplano) minimizando el error cuadrático medio
- Regresión logística: la misma idea, pasada por una sigmoide, para obtener una probabilidad
- Ambas se entrenan con descenso de gradiente y comparten la interfaz `fit` / `predict` de `scikit-learn`

¿Preguntas?